

Trajectory に基づく脆弱性対応

ログイン後セキュリティ監視のための GyroAuth 応用モデル

Gyro Logic Lab

2026

Author: Gyro Logic Lab

Repository: <https://github.com/gitGyro-Dev/gyroauth>

Applied PoC: [examples/trajectory_vulnerability_response/](#)

要旨

AI支援による脆弱性探索が高度化する中で、既知の脆弱性を事前に防止または修正することだけに依存したセキュリティモデルは不十分になる可能性がある。本稿では、ログイン後セキュリティ監視のための GyroAuth 応用モデルとして、Trajectory-Based Vulnerability Response を提案する。本稿の中心的主張は、攻撃が個別には正常で許可されたイベントから構成される場合であっても、その連なりとして本来許容されない不安定な操作Trajectoryを形成し得るという点にある。本モデルでは、脆弱性を「本来許容されないTrajectoryを正常処理として受理してしまう構造的弱点」として捉える。提案モデルは、予防策としての Boundary / Negative Definition、通過後の異常遷移を検知する Trajectory Detection、被害限定・再認証・隔離・回復を行う Stability Response の三層から構成される。最小PoCでは、合成アクセスログからセッションTrajectoryを再構成し、順序、速度、文脈、範囲、量、権限、到達先のDeviationを評価し、重み付きDeviationからStabilityを算出し、段階的Responseへ写像する。本稿は脆弱性の自動発見や攻撃再現を目的とするものではなく、GyroAuth をログイン時の本人確認から継続的なTrajectory収束監視へ拡張する概念実証である。

キーワード: GyroAuth, Gyro Logic, Trajectory検知, 脆弱性対応, ログイン後セキュリティ, Stability Response, 状態収束, サイバーセキュリティ, 認証, 異常検知

1. はじめに

AIによる脆弱性探索支援能力の向上は、セキュリティ設計に対して新たな課題を提示している。未知の弱点が従来より高速に発見される状況では、すべての脆弱性を事前に発見し修正することだけに依存した防御は十分ではなくなる可能性がある。

これは、予防・パッチ適用・入力検証・アクセス制御が不要であることを意味しない。それらは依然として重要である。しかし現代のシステムには、それに加えて、正常処理経路を通過した後の異常Trajectoryを検知し応答する層が必要になる。

多くの攻撃は、単一の異常イベントとしては現れない。ログインは成功し、APIアクセスは200 OKを返し、ダウンロードも正常に完了する場合がある。問題はイベント単体ではなく、その順序、速度、範囲、文脈、到達先によって形成されるTrajectoryにある。

本稿では、以下の区別を中心に据える。

Trajectory

より短く言えば：

`valid events != stable trajectory`

本稿は、この考えを GyroAuth の応用モデルとして整理する。

2. GyroAuth の背景

GyroAuth は、Gyro Logic を理論基盤とし、GyroOS に関連して実装される応用レイヤである。その中心定義は以下である。

Authentication = Stability-based Selection over State Convergence

一般的な認証システムでは、認証は時点的判定として扱われることが多い。ユーザーが資格情報を入力し、トークンが発行され、セッションは認証済みとして扱われる。

GyroAuth はこれを状態過程として扱う。多次元状態が期待される Identity Trajectory へ収束しているかを観測し、その安定性、Deviation、Operator Response、履歴に基づいて認証状態を選択する。

本稿では、この考えをログイン時の本人確認から、ログイン後セッションの Trajectory 監視へ拡張する。

問題は単に：

ではなく：

Trajectory Identity

である。

本稿は Gyro Logic の再定義ではなく、GyroAuth の応用ノートである。

3. 本来許容されない Trajectory としての脆弱性

脆弱性は一般に、ソフトウェア・設定・設計・運用における弱点として説明される。本モデルでは、これを Trajectory の観点から次のように定義する。

Trajectory

ここで重要なのは、脆弱性が単なるバグではないという点である。バグは期待された動作の失敗である。一方、脆弱性は、セキュリティ境界・状態遷移制約・運用前提を保持できないことにある。

例として以下がある。

- ・ データが実行コマンド空間へ侵入する
- ・ 未認証状態が認証 Trajectory へ侵入する
- ・ 一般ユーザー Trajectory が管理 Trajectory へ侵入する
- ・ resource 境界を越えて他者データへ到達する
- ・ 異常なデータ移動後もセッションが通常継続する

したがって GyroAuth 的には、イベントが許可されたかだけでなく、その結果形成される Trajectory が安定しているかを見る必要がある。

4. 正常イベントによって形成される不安定 Trajectory としての攻撃

攻撃は必ずしも単一の異常イベントとして現れない。実際には、攻撃者は正常操作を異常順序で並べる場合が多い。

例えば：

```
login
-> api_access
-> bulk_download
-> external_transfer
```

各イベントは成功している可能性がある。ログインは有効であり、API は到達可能であり、ダウンロード機能も存在し、外部送信も許可された endpoint 経由かもしれない。しかし Trajectory 全体としては、本来許容されない。

セッション Trajectory を以下で定義する。

$$T_s = (e_1, e_2, \dots, e_n)$$

ここで e_i は session s に属するイベントである。

イベント単体では正当性が成立し得る。

$$\forall e_i \in T_s, \text{allow}(e_i) = \text{true}$$

しかし Trajectory の安定性は崩壊し得る。

$$\text{Stable}(T_s \mid C_s) < \theta$$

ここで C_s は user, role, device, time, location, resource, history を含む文脈である。

本稿の中心主張は以下である。

????????????????Trajectory????????????

5. 三層モデル

提案モデルは、Figure 1 に示す三層から構成される。

Trajectory-Based Vulnerability Response は以下の三層で構成される。

Boundary / Negative Definition

Trajectory Detection

Stability Response

これは予防・検知・応答に対応する。

Figure 1. Three-Layer Model

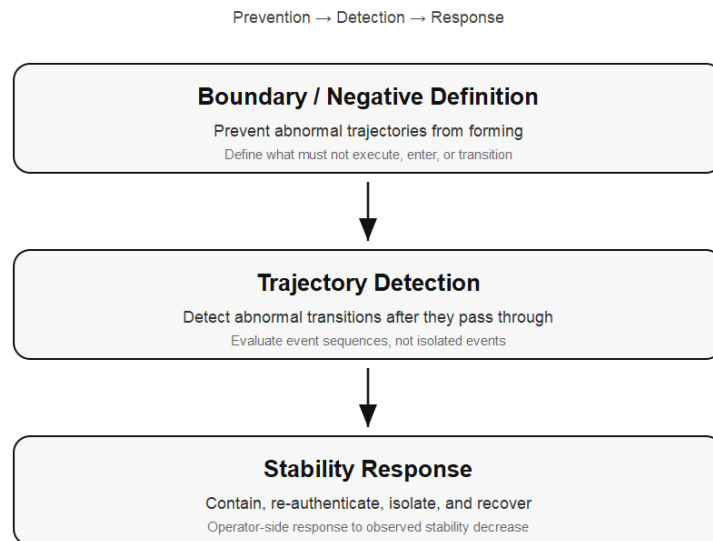


図 1: Figure 1. 三層モデル：予防、検知、応答。

5.1 Boundary / Negative Definition

Boundary / Negative Definition は予防層である。

セキュリティ設計は、「何を実行するか」の定義だけでは不十分である。

以下も定義する必要がある。

- 何を実行してはならないか
- どの状態遷移を許容しないか
- どの入力を command space に入れてはならないか
- どの role が resource 境界を越えてはならないか
- どの不安定状態が critical operation に入ってはならないか

Boundary / Negative Definition は、この意味で異常Trajectoryが形成される空間を狭める。

5.2 Trajectory Detection

Trajectory Detection は通過後検知層である。

どの境界も完全ではないため、一部の異常Trajectoryは通常制御を通過し得る。Trajectory Detection はログからイベント列を再構成し、期待される文脈の中で安定しているかを評価する。

主なDeviationは以下である。

- sequence deviation
- velocity deviation
- context deviation
- scope deviation
- volume deviation
- permission deviation

- destination deviation

検知対象はイベント単体ではなくイベント列である。

Detection target = trajectory, not isolated event

5.3 Stability Response

Stability Response は応答層である。

検知だけでは防御にならない。Trajectory Deviation が観測されたとき、検知結果を段階的制御へ接続する必要がある。

応答例は以下である。

- continue
- observe
- challenge
- restrict
- isolate
- recover

ここで重要なのは、Stability 自体が判断主体ではない点である。Stability は観測される状態変数であり、Operator-side response function が Stability と Deviation をもとに応答を選択する。

$$R = \rho(S, \Delta, C)$$

6. 既存システムへの後付けアーキテクチャ

提案モデルの後付け構成を Figure 2 に示す。既存システムから出力されるログやイベントを Slice として正規化し、それらをセッション単位の Trajectory として再構成する。その後、Deviationを検出し、Stabilityを評価し、必要に応じて Response Controller が既存の制御点へ応答を返す。

Figure 2. Existing-System Overlay Architecture

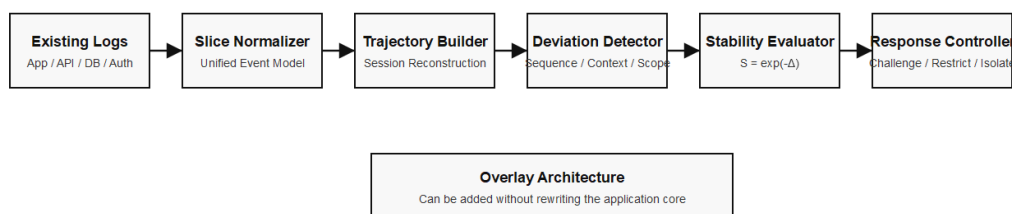


図 2: Figure 2. 既存システムに後付けするTrajectory監視アーキテクチャ。

提案モデルは、既存システムを書き換えずに追加可能である。最小構造は以下である。

Existing Logs

- > Slice Normalizer
- > Trajectory Builder
- > Stability Evaluator
- > Deviation Detector
- > Response Controller

システムはまず既存環境からログ・イベントを収集する。対象には application logs, API gateway logs, authentication logs, authorization logs, database audit logs, file access logs, session logs などが含まれる。

Slice Normalizer はこれらを共通イベント形式へ変換する。

Trajectory Builder は session, user, device, resource, time window ごとにTrajectoryを構築する。

Stability Evaluator は通常Trajectoryとの差分を評価する。

Response Controller は IAM, MFA, API Gateway, rate limiter, SIEM, session manager などの既存制御点を用いて応答する。

7. PoC

最小PoCは以下に実装されている。

`examples/trajectory_vulnerability_response/`

PoCは synthetic JSONL access logs を用い、以下の三種類のセッションを含む。

1. normal session
2. suspicious session
3. attack-like session

PoCは以下の流れを実装する。

Logs -> Slice -> Trajectory -> Deviation -> Stability -> Response

7.1 シナリオ

正常セッション：

`login -> view_dashboard -> search -> view_record -> logout`

疑わしいセッション：

`login -> api_access -> export`

攻撃的セッション：

`login -> api_access -> bulk_download -> external_transfer`

すべてのイベントは **success** として記録される。これは意図的である。本PoCは、「正常に成功したイベント列」が不安定Trajectoryを形成し得ることを示す。

7.2 Stability 計算

PoCでは、以下のDeviationを評価する。

- sequence
- velocity
- context
- scope
- volume
- permission
- destination

重み付きDeviationを以下で計算する。

$$\Delta(T_s) = w_{\text{seq}}\Delta_{\text{seq}} + w_{\text{vel}}\Delta_{\text{vel}} + w_{\text{ctx}}\Delta_{\text{ctx}} + w_{\text{scope}}\Delta_{\text{scope}} + w_{\text{vol}}\Delta_{\text{vol}} + w_{\text{perm}}\Delta_{\text{perm}} + w_{\text{dest}}\Delta_{\text{dest}}$$

Stability は以下で計算する。

$$S(T_s) = \exp(-\Delta(T_s))$$

これは PoC 用の実装モデルであり、Gyro Logic 本体の再定義ではない。

7.3 Response Mapping

PoCは Stability を段階的応答へ写像する。

Stability	Auth State	Response
$S \geq 0.85$	AUTH_STABLE	continue
$S \geq 0.65$	RECONVERGING	observe
$S \geq 0.45$	AUTH_CHALLENGE	challenge
$S \geq 0.25$	AUTH_RESTRICTED	restrict
$S < 0.25$	AUTH_ISOLATED	isolate

7.4 結果

Figure 3 は、PoCにおける各セッションのStability結果を示す。正常セッションでは Stability が 1.0000 に保たれ、AUTH_STABLE / continue が選択された。一方、疑わしいセッションでは Stability が 0.6219 まで低下し、AUTH_CHALLENGE / challenge が選択された。攻撃的セッションでは Stability が 0.0669 まで低下し、AUTH_ISOLATED / isolate が選択された。

PoCは以下の結果を生成した。

Session	Delta	Stability	Auth State	Response
normal	0.000	1.0000	AUTH_STABLE	continue
suspicious	0.475	0.6219	AUTH_CHALLENGE	challenge
attack-like	2.705	0.0669	AUTH_ISOLATED	isolate

Figure 3. PoC Stability Results

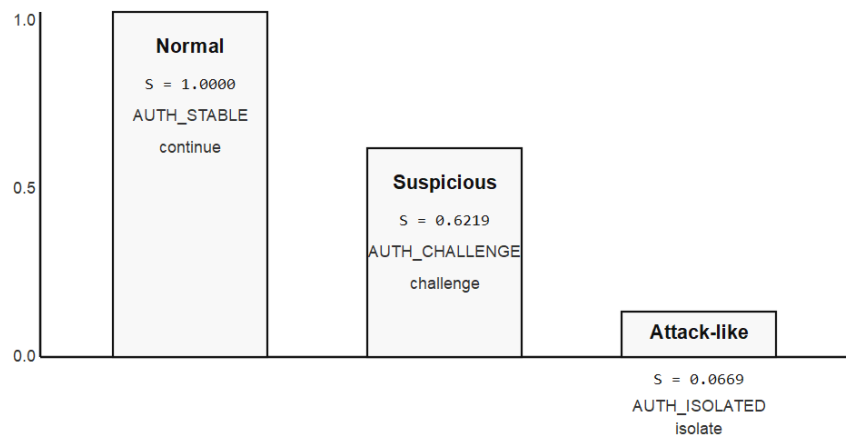


図 3: Figure 3. 正常・疑わしい・攻撃的セッションに対するPoC Stability結果。

正常セッションは安定状態を維持した。疑わしいセッションは MFA と device verification を伴う challenge を要求した。攻撃的セッションは session isolation, token revocation, external transfer blocking, audit log preservation, administrator notification を含む isolate 応答を返した。

これは本PoCの中心主張を示している。

Trajectory

8. 議論

本モデルは intrusion detection, SIEM, EDR, UEBA など既存監視概念と近い目的を持つ。しかし本稿の焦点は異なる。本モデルはイベント単体を normal / abnormal に分類するだけでなく、イベント列をTrajectoryとして再構成し、それが Identity, permission, resource, operational context の中で安定しているかを見る。

また、本モデルは既存システムと共存可能である。ログベース監視から始め、alerting, step-up authentication, dynamic restriction, session isolation へ段階的に発展できる。

Deviation は必ずしも攻撃を意味しない。ユーザーは移動し、新端末を利用し、大量業務を行う場合がある。したがって Stability Response は即時遮断ではなく、段階的 Operator-side response である必要がある。

9. 制限事項

本稿は概念モデルおよび最小PoCであり、以下の制限を持つ。

- synthetic logs を用いている
- 実データセット評価を行っていない
- threshold および weight は例示的である
- production IDS claim を行わない
- false positive 調整が必要である

- 応答には実システムとの統合が必要である
- patching, secure coding, access control, existing security tools を置き換えるものではない

本PoCの目的は exploit を再現することではない。正常イベント列から不安定Trajectoryを検知し、段階的Responseへ接続できることを示す点にある。

10. 結論

本稿では、Trajectory-Based Vulnerability Response を GyroAuth の応用モデルとして提案した。本モデルは、脆弱性悪用が個別には正常なイベントから構成されながら、本来許容されないTrajectoryを形成し得るという観察から出発する。

提案モデルは以下の三層から構成される。

Boundary / Negative Definition = 図2

Trajectory Detection = 図3

Stability Response = 図4

最小PoCは、正常に成功したイベント列を session trajectory として再構成し、Deviation を Stability に変換し、段階的Responseへ接続可能であることを示した。

本稿の中心結論は以下である。

図5 Trajectory図6

AI支援による脆弱性探索と複雑化するシステム環境において、脆弱性対応はイベント単体の許可可否だけではなく、そのイベント列が形成するTrajectory全体の安定性を評価する必要がある。

利益相反

著者は、本稿に関して開示すべき利益相反はない。

参考文献

1. Gyro Logic Lab, Gyro Logic: A Stability-Based Framework for Representation Under Intrinsic Deviation, Jxiv, DOI: <https://doi.org/10.51094/jxiv.4159>
2. Gyro Logic Lab, Gyro Logic: A Stability-Based Framework for Representation Under Intrinsic Deviation Japanese translation, Jxiv, DOI: <https://doi.org/10.51094/jxiv.4597>
3. Gyro Logic Lab, GyroAuth, GitHub repository: <https://github.com/gitGyro-Dev/gyroauth>
4. Gyro Logic Lab, Trajectory-Based Vulnerability Response PoC, GitHub repository path: https://github.com/gitGyro-Dev/gyroauth/tree/main/examples/trajectory_vulnerability_response